

Entrega UT4

Actividad 1:Ventajas de la modularización

Tarea:

Redacta un texto breve explicando las ventajas de la modularización en componentes personalizados. Incluye un ejemplo en JavaScript.

La modularización en componentes personalizados permite tener tus propios componentes los cuales serán exactamente como quieras, se verán siempre igual independientemente que entres en un navegador u otro y también los podras reutilizar para otros proyectos simplemente importando el script.

Ejemplo:

```
class MyButton extends HTMLElement//HTMLElement es lo que tiene todo
{
  constructor() { //
    super(); //inicializa elementos del padre
    const color = this.getAttribute('color') || 'red'; //pilla atributo del padre
    const nombre = this.getAttribute('nombre');
    const shadow = this.attachShadow({mode: 'open'});
    const button = document.createElement('button');//va al dom y le crea elemento button
    button.textContent = this.textContent; //contenido del padre

    const style = document.createElement('style');
    style.textContent = `
      button {
        background-color: ${color};
        color: white; /* color texto */
        border: none;
        padding: 10px 20px;
        border-radius: 8px;
        cursor: pointer;
        font-size: 16px;
        transition: background-color 0.3s;
      }

      button:hover {
        background-color: #4338ca;
      }
    `; // al pasar por encima le cambia el background

    shadow.appendChild(style); //le mete el estilo y el boton
    shadow.appendChild(button);

    button.addEventListener('click', () => {
      alert(`Has hecho click en "${button.textContent}, ${nombre}"`);
    });
  }
}

customElements.define('my-button', MyButton); //añadimos
```

Y en el html lo importamos

```
<script src="MyButton.js" defer></script>
```

y creamos uno

```
<my-button color="blue", nombre="Jose">Botón Personalizado</my-button>
```

Actividad 2: Diseño de componente con propiedades y eventos

Tarea:

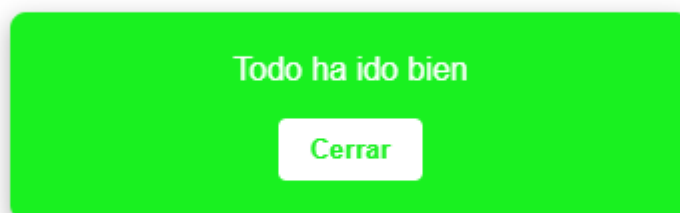
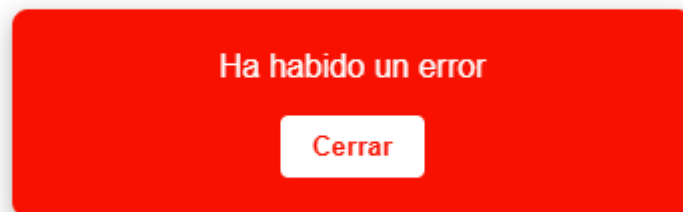
Diseña un componente personalizado (puede ser ficticio) y describe:

- 2 propiedades configurables (nombre, tipo, valor por defecto)
- 2 eventos gestionables (nombre, cuándo se activan)

He creado una alerta.(El código lo pondré en la siguiente actividad)

Las dos propiedades configurables que tengo son el tipo de alerta (según que tipo la alerta tiene un color u otro) y el mensaje(Cambia el mensaje de la alerta).

```
this.tipoDeAlerta = this.getAttribute("tipoDeAlerta") || "informar";  
this.mensaje = this.getAttribute("mensaje") || "No hay mensaje";
```



El primer evento que tengo es que nada mas abrir la página se llame al metodo Render que he creado donde renderizo la alerta.

```
connectedCallback()
{
  this.Renderizar(); //llamo al metodo render
}
```

Y el segundo que tengo es que la alerta tiene un boton que cuando clico se cierra

```
<div class="alerta">
  <span>${this.mensaje}</span>
  <button id="cerrar">Cerrar</button>
</div>
`; //creo un div donde pondre el mensaje de la alerta y un boton para cerrar

this.shadowRoot.querySelector('#cerrar').addEventListener('click', () => {
  this.remove(); //elimino el componente del DOM
});
```

Actividad 3: Pruebas individuales y documentación técnica

Tarea:

Simula que has creado un componente.

Describe:

- 3 pruebas individuales realizadas (visualización, interacción, errores)
- Documentación técnica básica: descripción, propiedades, eventos, ejemplo de uso

html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Ejemplo</title>
  <script src="MyAlert.js" defer></script>
</head>
<body>
  <my-alert tipoDeAlerta="success" mensaje="Todo ha ido bien"></my-alert>
</body>
</html>
```

js

```
class MyAlert extends HTMLElement{
  constructor(){
    super();
    this.attachShadow({ mode: 'open' });
    this.tipoDeAlerta = this.getAttribute("tipoDeAlerta") || "informar";
    this.mensaje = this.getAttribute("mensaje") || "No hay mensaje";
  }

  connectedCallback()
  {
    this.Renderizar(); //llamo al metodo render
  }

  Renderizar()
  {
    const colores = { //creo una lista con los colores
      success: '■ #19f120ff',
      error: '■ #f81100ff',
      warning: '■ #c48323ff'
    };

    //shadow creado antes con attachshaadow
    this.shadowRoot.innerHTML = `
      <style>
        .alerta {
          position: fixed; /* se mantiene en la pantalla */
          top: 50%; /* centro vertical */
          left: 50%; /* centro horizontal */
          transform: translate(-50%, -50%); /* ajuste perfecto para centrar */
          width: 300px; /* tamaño fijo para el cuadro */
          padding: 20px;
          border-radius: 8px; /* bordes ligeramente redondeados */
          background-color: ${colores[this.tipoDeAlerta]};
          color: white;
          display: flex;
          flex-direction: column; /* vertical: mensaje arriba, botón abajo */
          align-items: center;
          font-family: sans-serif;
          text-align: center;
          box-shadow: 0 4px 12px □ rgba(0,0,0,0.3); /* sombra para destacar */
          z-index: 9999; /* asegurarse que esté encima de todo */
        }
      </style>
    `;
  }
}
```

```

        button {
            margin-top: 15px;
            padding: 8px 16px;
            background: white;
            color: ${colores[this.tipoDeAlerta]};
            border: none;
            border-radius: 4px;
            font-weight: bold;
            cursor: pointer;
        }
    </style>

    <div class="alerta">
        <span>${this.mensaje}</span>
        <button id="cerrar">Cerrar</button>
    </div>
    `; //creo un div donde pondre el mensaje de la alerta y un boton para cerrar

    this.shadowRoot.querySelector('#cerrar').addEventListener('click', () => {
        this.remove(); //elimino el componente del DOM
    });
}

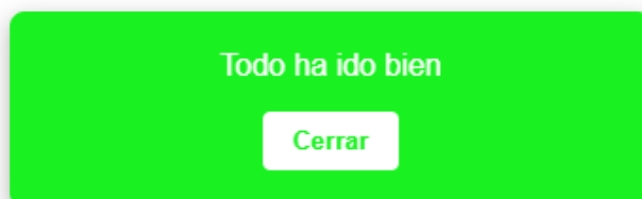
customElements.define('my-alert', MyAlert); //añado a customElements

```

Al principio el componente se veía muy feo ocupando una línea diagonal



Así que modifiqué el estilo para que pareciera más una alerta



Sobre la interacción he hecho que nada más entres a la página aparezca la alerta

```

connectedCallback()
{
    this.Renderizar(); //llamo al metodo render
}

```

Y también que tenga el botón cerrar que permite cerrarla

```

<div class="alerta">
  <span>${this.mensaje}</span>
  <button id="cerrar">Cerrar</button>
</div>
`; //creo un div donde pondre el mensaje de la alerta y un boton para cerrar

this.shadowRoot.querySelector('#cerrar').addEventListener('click', () => {
  this.remove(); //elimino el componente del DOM
});

```

Y finalmente había cometido varios errores como que sin querer la variable “tipoDeAlerta” luego cuando he querido poner el color en el shadowRoot había puesto “tipo” entonces no iba bien.

```
background-color: ${colores[this.tipo]};
```

Y otro que he cometido es meter

```
customElements.define('my-alert', MyAlert); //añado a customElements
```

Dentro del método Renderizar sin querer pensando que estaba fuera, lo que me ha llevado un rato a no entender porque el editor me marcaba mal.

Las imágenes del código contienen documentación de cada parte del proceso.

Actividad 4: Empaquetado e integración del componente

Tarea:

Explica cómo empaquetar e importar un componente en otro proyecto:

- ¿Qué formato usarías? (.jar, npm, etc.)
- ¿Cómo lo integrarías en otro entorno?
- ¿Qué problemas podrías encontrar?

Como en el ejemplo que he realizado simplemente tienes que tener el archivo en este caso un js o si es para un proyecto más grande lo puedes empaquetar.

Después tienes que importar el script en tu html con:

```
<script src="MyAlert.js" defer></script>
```

Un problema que puedes encontrar es que a lo mejor lo has guardado en otra carpeta y no lo encuentras. Por tanto asegurarse que la ruta en src es la correcta.

Otro puede ser que a lo mejor este script depende de otro el cual no has importado y eso genere un conflicto. En este caso no pasaría porque este componente simplemente depende de `HTMLElement`